

UAS

PENGOLAHAN CITRA



NAMA : Muh. Fadil Imamsyah

NIM : 202431147

KELAS : A

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 8

ASISTEN : 1. Sharira Maharani

2. Muhammad Hanif Nuruddin Jabbar

3. Fhazel Kesra Arivi

4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah.....	3
1.2 Tujuan Masalah.....	3
1.3 Manfaat Masalah.....	3
BAB II.....	4
LANDASAN TEORI.....	4
2.1 Filtering Citra.....	4
2.2 Mean Filtering.....	4
2.3 Median Filtering.....	5
2.4 Implementasi Filtering Menggunakan OpenCV dan Python.....	5
2.5 Perbandingan Mean Filtering dan Median Filtering.....	6
BAB III.....	7
HASIL.....	7
3.1 Import Library.....	7
3.2 Membaca Gambar.....	7
3.3 Menampilkan Gambar Asli.....	8
3.4 Media filtering.....	8
3.5 Mean filtering secara manual.....	9
3.6 Menjalankan fungsi mean.....	9
3.7 Menampilkan Hasil I.....	10
3.8 Menampilkan hasil II.....	11
BAB VI.....	12
PENUTUP.....	12
4.1 Kesimpulan.....	12
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Apa yang dimaksud dengan filtering citra dan apa tujuan penerapannya dalam pengolahan citra digital?
2. Bagaimana prinsip kerja Mean Filtering dan Median Filtering sebagai metode filtering citra?
3. Bagaimana implementasi Median Filtering menggunakan pustaka OpenCV pada citra digital?
4. Bagaimana implementasi Mean Filtering secara manual menggunakan perhitungan rata-rata nilai piksel pada kernel?
5. Bagaimana hasil dan perbandingan citra setelah diterapkan Mean Filtering dan Median Filtering?

1.2 Tujuan Masalah

1. Memahami konsep dasar filtering citra dalam pengolahan citra digital.
2. Memahami prinsip kerja Mean Filtering dan Median Filtering.
3. Mengimplementasikan Median Filtering menggunakan OpenCV.
4. Mengimplementasikan Mean Filtering secara manual menggunakan Python.
5. Menganalisis dan membandingkan hasil kedua metode filtering terhadap kualitas citra.

1.3 Manfaat Masalah

1. Menambah pemahaman mengenai konsep filtering citra digital.
2. Memahami karakteristik Mean Filtering dan Median Filtering.
3. Meningkatkan kemampuan mengimplementasikan algoritma filtering menggunakan Python dan OpenCV.
4. Mengetahui pengaruh filtering terhadap kualitas citra digital.
5. Menjadi dasar dalam mempelajari teknik peningkatan kualitas citra pada pengolahan citra digital.

BAB II

LANDASAN TEORI

2.1 Filtering Citra

Filtering citra merupakan salah satu teknik dasar dalam pengolahan citra digital yang digunakan untuk memodifikasi nilai intensitas piksel dengan tujuan meningkatkan kualitas visual suatu citra. Proses filtering dilakukan dengan menerapkan suatu filter atau kernel pada setiap piksel citra sehingga dapat mengurangi gangguan (noise), memperhalus citra, maupun mempertajam informasi tertentu sesuai kebutuhan analisis. Filtering umumnya menjadi tahapan pre-processing sebelum citra diproses lebih lanjut, seperti segmentasi, ekstraksi fitur, maupun pengenalan objek.

Menurut Gonzalez dan Woods (2018), filtering citra merupakan operasi spasial yang memproses nilai piksel berdasarkan hubungan dengan piksel-piksel di sekitarnya menggunakan sebuah kernel atau mask. Operasi ini bertujuan menghasilkan citra dengan kualitas yang lebih baik sehingga informasi yang terkandung di dalamnya dapat dianalisis secara lebih akurat.

Penelitian yang dilakukan oleh Singh dan Sharma (2020) menunjukkan bahwa proses filtering memiliki peran penting dalam meningkatkan kualitas citra digital sebelum dilakukan proses pengenalan pola maupun computer vision. Hasil penelitian tersebut menjelaskan bahwa pemilihan metode filtering yang tepat mampu mengurangi noise tanpa menghilangkan informasi penting pada objek.

Berdasarkan prinsip kerjanya, filtering dibedakan menjadi dua kelompok utama yaitu linear filtering dan non-linear filtering. Mean Filtering termasuk ke dalam linear filtering karena prosesnya menggunakan nilai rata-rata piksel, sedangkan Median Filtering merupakan non-linear filtering karena menggunakan nilai median dari piksel tetangga.

2.2 Mean Filtering

Mean Filtering merupakan metode linear smoothing filter yang bekerja dengan mengganti nilai suatu piksel menggunakan nilai rata-rata dari seluruh piksel di dalam kernel. Tujuan utama metode ini adalah mengurangi noise acak (random noise) serta menghasilkan citra yang lebih halus. Metode Filtering pada Citra Digital

Secara matematis, Mean Filtering dirumuskan sebagai berikut.

$$g(x, y) = \frac{1}{M \times N} \sum_{i=-a}^a \sum_{j=-b}^b f(x + i, y + j)$$

Dengan:

- $f(x, y)$ = citra masukan
- $g(x, y)$ = citra hasil filtering
- $M \times N$ = ukuran kernel

Pada praktikum ini digunakan kernel berukuran 3×3 , sehingga setiap piksel dihitung berdasarkan rata-rata sembilan piksel yang berada di sekitarnya.

Menurut penelitian Kaur dan Kaur (2019), Mean Filtering mampu mengurangi noise Gaussian dengan cukup baik karena proses perataan nilai piksel menghasilkan citra yang lebih halus. Namun, metode ini memiliki kelemahan yaitu menyebabkan detail objek dan tepi citra menjadi kurang tajam akibat seluruh nilai piksel dirata-ratakan.

Pada implementasi praktikum, Mean Filtering tidak menggunakan fungsi bawaan OpenCV, melainkan dibuat secara manual menggunakan proses perhitungan rata-rata setiap kernel. Implementasi tersebut bertujuan agar mahasiswa memahami mekanisme kerja algoritma Mean Filtering secara langsung.

2.3 Median Filtering

Median Filtering merupakan metode non-linear filtering yang mengganti nilai suatu piksel menggunakan nilai median dari seluruh piksel dalam kernel. Berbeda dengan Mean Filtering, metode ini tidak menggunakan proses rata-rata sehingga lebih efektif dalam menghilangkan noise impulsif, seperti Salt and Pepper Noise.

Proses Median Filtering dilakukan melalui beberapa tahapan, yaitu mengambil seluruh nilai piksel pada kernel, mengurutkan nilai tersebut dari yang terkecil hingga terbesar, kemudian memilih nilai tengah sebagai pengganti nilai piksel pusat.

Menurut Gonzalez dan Woods (2018), Median Filtering mampu mempertahankan tepi objek dengan lebih baik dibandingkan filter linear karena nilai median tidak dipengaruhi oleh nilai piksel yang terlalu tinggi maupun terlalu rendah.

Hasil penelitian Sharma et al. (2021) juga menunjukkan bahwa Median Filtering menghasilkan kualitas citra yang lebih baik pada citra yang mengandung noise impulsif dibandingkan Mean Filtering. Hal tersebut dibuktikan melalui peningkatan kualitas citra berdasarkan parameter Peak Signal-to-Noise Ratio (PSNR) dan Mean Square Error (MSE).

Pada praktikum ini, Median Filtering diimplementasikan menggunakan fungsi `cv2.medianBlur()` yang tersedia pada pustaka OpenCV. Fungsi tersebut secara otomatis menghitung nilai median pada setiap kernel sesuai ukuran yang ditentukan sehingga proses filtering dapat dilakukan secara efisien.

2.4 Implementasi Filtering Menggunakan OpenCV dan Python

Python merupakan bahasa pemrograman yang banyak digunakan dalam bidang pengolahan citra digital karena memiliki sintaks yang sederhana dan didukung berbagai pustaka seperti OpenCV, NumPy, dan Matplotlib. OpenCV menyediakan berbagai fungsi pengolahan citra, salah satunya adalah `medianBlur()` yang digunakan untuk menerapkan Median Filtering.

Pada praktikum ini, implementasi filtering dilakukan menggunakan dua pendekatan. Median Filtering diterapkan menggunakan fungsi bawaan OpenCV sehingga proses filtering dilakukan secara otomatis berdasarkan ukuran kernel yang ditentukan. Sebaliknya, Mean Filtering dibuat secara manual menggunakan operasi perulangan (loop) dan perhitungan rata-rata setiap piksel pada kernel berukuran 3×3 . Pendekatan ini bertujuan memberikan pemahaman mengenai cara kerja algoritma filtering secara langsung tanpa bergantung pada fungsi bawaan pustaka.

Menurut Bradski dan Kaehler (2008), OpenCV dirancang untuk menyediakan implementasi algoritma pengolahan citra yang efisien dan mudah digunakan sehingga banyak dimanfaatkan dalam penelitian maupun pengembangan aplikasi computer vision.

2.5 Perbandingan Mean Filtering dan Median Filtering

Mean Filtering dan Median Filtering sama-sama digunakan untuk mengurangi noise pada citra digital, tetapi memiliki prinsip kerja yang berbeda. Mean Filtering menggunakan nilai rata-rata seluruh piksel dalam kernel sehingga menghasilkan efek smoothing yang cukup tinggi. Sebaliknya, Median Filtering menggunakan nilai median sehingga mampu mempertahankan detail objek dengan lebih baik.

Menurut penelitian Khan et al. (2022), Mean Filtering lebih sesuai digunakan pada citra yang mengandung noise acak (Gaussian noise), sedangkan Median Filtering lebih efektif dalam menghilangkan noise impulsif (Salt and Pepper Noise). Oleh karena itu, pemilihan metode filtering harus disesuaikan dengan karakteristik noise pada citra yang akan diproses.

Berdasarkan implementasi pada praktikum ini, Mean Filtering menghasilkan citra yang tampak lebih halus, tetapi beberapa detail objek mengalami penurunan ketajaman. Sebaliknya, Median Filtering mampu mengurangi noise tanpa menghilangkan detail penting pada objek sehingga hasil citra terlihat lebih alami. Perbedaan karakteristik tersebut menunjukkan bahwa setiap metode memiliki kelebihan dan kekurangan sesuai tujuan pengolahan citra yang diinginkan.

BAB III

HASIL

3.1 Import Library

IMPORT LIBRARY

```
1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Gambar 3. 1 Import Library

Pada tahap awal program dilakukan proses import library untuk memanggil pustaka yang dibutuhkan dalam pengolahan citra digital. Library OpenCV (cv2) digunakan untuk membaca, memproses, dan menerapkan Median Filtering pada citra. Library NumPy (numpy) digunakan untuk melakukan operasi numerik dan manipulasi array, khususnya dalam proses Mean Filtering secara manual melalui perhitungan rata-rata nilai piksel. Sementara itu, Matplotlib (matplotlib.pyplot) digunakan untuk menampilkan citra asli maupun citra hasil filtering sehingga memudahkan proses analisis dan perbandingan hasil pengolahan citra.

3.2 Membaca Gambar

MEMBACA GAMBAR

```
[2]: image = cv2.imread("fadil.jpeg")
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Gambar 3. 2 Code Membaca Gambar

Kode tersebut digunakan untuk membaca citra yang akan diolah serta mengubah format warna sesuai kebutuhan proses pengolahan citra. Baris `cv2.imread("fadil.jpeg")` berfungsi untuk membaca file gambar `fadil.jpeg` dan menyimpannya ke dalam variabel `image`. Selanjutnya, `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` digunakan untuk mengubah format warna citra dari BGR (Blue, Green, Red), yang merupakan format bawaan OpenCV, menjadi RGB (Red, Green, Blue) agar warna gambar ditampilkan dengan benar menggunakan Matplotlib. Terakhir, `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)` digunakan untuk mengonversi citra berwarna menjadi grayscale, yaitu citra dengan satu kanal intensitas keabuan, sehingga memudahkan proses filtering dan pengolahan citra pada tahap berikutnya.

3.3 Menampilkan Gambar Asli

```
[3]: plt.figure(figsize=(6,6))
plt.imshow(image_rgb)
plt.title("Citra Asli")
plt.axis("on")
plt.show()
```



Gambar 3. 3 code Menampilkan Gambar Asli

Kode di atas digunakan untuk menampilkan citra asli yang telah dikonversi ke format RGB menggunakan library Matplotlib. Fungsi `plt.figure(figsize=(6,6))` mengatur ukuran jendela tampilan menjadi 6×6 inci, sedangkan `plt.imshow(image_rgb)` menampilkan gambar yang tersimpan pada variabel `image_rgb`. Selanjutnya, `plt.title("Citra Asli")` memberikan judul pada gambar, `plt.axis("on")` menampilkan sumbu koordinat citra, dan `plt.show()` digunakan untuk menampilkan hasil akhir ke layar. Tahap ini bertujuan untuk memvisualisasikan citra asli sebagai acuan sebelum dilakukan proses filtering.

3.4 Media filtering

```
: median = cv2.medianBlur(image,5)
median_rgb = cv2.cvtColor(median,cv2.COLOR_BGR2RGB)
```

Gambar 3. 4 code media filtering

Kode di gunakan digunakan untuk menerapkan Median Filtering pada citra menggunakan fungsi bawaan OpenCV. Perintah `cv2.medianBlur(image, 5)` menerapkan filter median dengan ukuran kernel 5×5 , di mana setiap nilai piksel diganti dengan nilai median dari piksel-piksel di sekitarnya. Metode ini efektif untuk mengurangi noise, terutama salt and pepper noise, tanpa menghilangkan detail tepi objek secara signifikan. Selanjutnya, `cv2.cvtColor(median, cv2.COLOR_BGR2RGB)` digunakan untuk mengubah format warna hasil filtering dari BGR menjadi RGB agar warna citra dapat ditampilkan dengan benar menggunakan Matplotlib.

3.5 Mean filtering secara manual

MEAN FILTERING SECARA MANUAL

```
def mean_filter_manual(img, kernel=3):
    tinggi, lebar = img.shape
    hasil = np.zeros_like(img)
    pad = kernel // 2
    padded = np.pad(img,
                     ((pad,pad),(pad,pad)),
                     mode='constant')
    for i in range(tinggi):
        for j in range(lebar):
            area = padded[i:i+kernel,
                          j:j+kernel]
            hasil[i,j] = np.mean(area)
    return hasil
```

Gambar 3. 5 code Mean Filtering Secara Manual

Kode merupakan fungsi `mean_filter_manual()` yang digunakan untuk menerapkan Mean Filtering secara manual pada citra grayscale dengan ukuran kernel 3×3 . Program diawali dengan mengambil ukuran citra (tinggi dan lebar), kemudian membuat matriks kosong sebagai tempat menyimpan hasil filtering menggunakan `np.zeros_like()`. Selanjutnya, citra diberikan padding menggunakan `np.pad()` agar proses filtering tetap dapat dilakukan pada piksel yang berada di tepi citra. Proses filtering dilakukan menggunakan dua buah perulangan (`for`) yang mengakses setiap piksel, kemudian mengambil area kernel berukuran 3×3 di sekitar piksel tersebut. Nilai setiap piksel hasil diperoleh dari rata-rata (`np.mean()`) seluruh nilai piksel dalam kernel, kemudian disimpan ke dalam matriks hasil. Setelah seluruh piksel diproses, fungsi mengembalikan citra hasil Mean Filtering melalui perintah `return hasil`.

3.6 Menjalankan fungsi mean

```
mean = mean_filter_manual(gray,3)
```

Gambar 3. 6 menjalankan fungsi mean

Baris kode `mean = mean_filter_manual(gray, 3)` digunakan untuk memanggil fungsi `mean_filter_manual()` yang telah dibuat sebelumnya untuk menerapkan Mean Filtering pada citra grayscale. Parameter `gray` merupakan citra masukan yang akan diproses, sedangkan angka 3 menunjukkan bahwa proses filtering menggunakan kernel berukuran 3×3 . Fungsi tersebut menghitung nilai rata-rata dari setiap piksel beserta piksel-piksel di sekitarnya, kemudian menggantikan nilai piksel asli dengan hasil rata-rata tersebut. Hasil proses filtering selanjutnya disimpan ke dalam variabel `mean` untuk digunakan pada tahap visualisasi dan analisis hasil pengolahan citra.

3.7 Menampilkan Hasil I

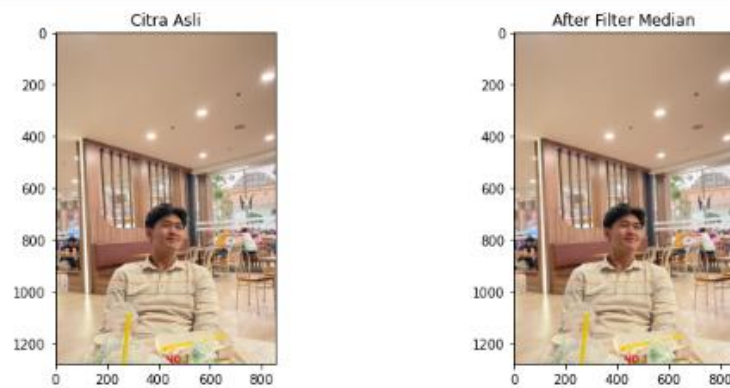
```
In [7]: plt.figure(figsize=(10,8))

# Citra Asli

plt.subplot(2,2,1)
plt.imshow(image_rgb)
plt.title("Citra Asli")
plt.axis("on")

# Median Filtering
plt.subplot(2,2,2)
plt.imshow(median_rgb)
plt.title("After Filter Median")
plt.axis("on")

plt.tight_layout()
plt.show()
```



Gambar 3. 7 Hasil dari Citra Asli dan After Filter Median

Kode tersebut digunakan untuk menampilkan citra asli dan citra hasil Median Filtering dalam satu jendela menggunakan library Matplotlib. Fungsi `plt.figure(figsize=(10,8))` mengatur ukuran tampilan gambar menjadi 10×8 inci, sedangkan `plt.subplot(2,2,1)` dan `plt.subplot(2,2,2)` membagi area tampilan menjadi empat bagian, di mana dua bagian pertama digunakan untuk menampilkan citra asli dan hasil filtering. Fungsi `plt.imshow()` digunakan untuk menampilkan masing-masing citra, `plt.title()` memberikan judul pada setiap gambar, dan `plt.axis("on")` menampilkan sumbu koordinat citra. Selanjutnya, `plt.tight_layout()` mengatur jarak antar gambar agar tidak saling bertumpuk, sedangkan `plt.show()` digunakan untuk menampilkan seluruh hasil visualisasi. Dari hasil yang diperoleh, citra setelah Median Filtering terlihat lebih halus dengan noise yang berkurang, namun detail dan tepi objek tetap terjaga sehingga kualitas citra menjadi lebih baik dibandingkan citra asli.

3.8 Menampilkan hasil II

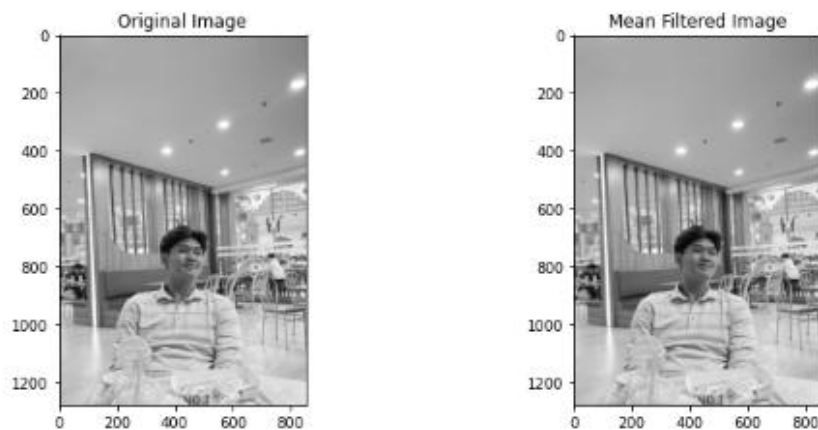
MENAMPILKAN HASIL II

```
[8]: plt.figure(figsize=(10,8))

# Original Gray
# -----
plt.subplot(2,2,3)
plt.imshow(gray,cmap='gray')
plt.title("Original Image")
plt.axis("on")

# Mean Filtering
plt.subplot(2,2,4)
plt.imshow(mean,cmap='gray')
plt.title("Mean Filtered Image")
plt.axis("on")

plt.tight_layout()
plt.show()
```



Gambar 3. 8 Menampilkan Hasil II

Kode di atas digunakan untuk menampilkan citra grayscale asli dan citra hasil Mean Filtering dalam satu jendela menggunakan library Matplotlib. Fungsi `plt.figure(figsize=(10,8))` mengatur ukuran tampilan gambar menjadi 10×8 inci, sedangkan `plt.subplot(2,2,3)` dan `plt.subplot(2,2,4)` digunakan untuk menempatkan citra grayscale asli dan citra hasil Mean Filtering pada posisi yang berbeda dalam satu tampilan. Fungsi `plt.imshow()` dengan parameter `cmap='gray'` menampilkan citra dalam skala keabuan, `plt.title()` memberikan judul pada masing-masing gambar, dan `plt.axis("on")` menampilkan sumbu koordinat citra. Selanjutnya, `plt.tight_layout()` mengatur jarak antar gambar agar tampilan lebih rapi, sedangkan `plt.show()` menampilkan hasil visualisasi. Berdasarkan hasil yang diperoleh, citra setelah Mean Filtering terlihat lebih halus karena nilai setiap piksel telah dirata-ratakan dengan piksel di sekitarnya, namun efek tersebut juga menyebabkan beberapa detail dan tepi objek tampak sedikit lebih kabur dibandingkan citra grayscale asli.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan yang telah dilakukan, dapat disimpulkan bahwa filtering citra merupakan salah satu teknik dalam pengolahan citra digital yang bertujuan untuk mengurangi noise dan meningkatkan kualitas citra. Pada praktikum ini, Median Filtering berhasil diimplementasikan menggunakan fungsi `cv2.medianBlur()` dari OpenCV, sedangkan Mean Filtering dibuat secara manual dengan menghitung rata-rata nilai piksel pada kernel berukuran 3×3 . Hasil pengujian menunjukkan bahwa kedua metode mampu menghasilkan citra yang lebih halus dibandingkan citra asli. Namun, Median Filtering lebih efektif dalam mengurangi noise tanpa menghilangkan detail dan tepi objek secara signifikan, sedangkan Mean Filtering memberikan efek penghalusan yang lebih besar sehingga beberapa detail citra menjadi sedikit kabur. Dengan demikian, pemilihan metode filtering perlu disesuaikan dengan karakteristik citra dan tujuan pengolahan yang ingin dicapai.

DAFTAR PUSTAKA

- Bradski, G., & Kaehler, A. (2016). *Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library*. O'Reilly Media. <https://www.oreilly.com/library/view/learning-opencv-3/9781491937983/>
- Fan, L., Zhang, F., Fan, H., & Zhang, C. (2019). Brief review of image denoising techniques. *Visual Computing for Industry, Biomedicine, and Art*, 2(7). <https://doi.org/10.1186/s42492-019-0016-7>
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/digital-image-processing/P2000000003224>
- Kong, Z., et al. (2023). *A Comparison of Image Denoising Methods*. arXiv. <https://arxiv.org/abs/2304.08990>
- Kumar, V., & Samadhiya, A. (2019). *Comparative Performance Analysis of Image Denoising Techniques*. arXiv. <https://arxiv.org/abs/1901.06529>